Computer Networks - Tutorial sheet 1

Q1. Data communication through the OSI model

The Open Systems Interconnection model describes communication in seven layers. At the sender, data moves from layer 7 down to layer 1; at the destination, it moves upward from layer 1 to layer 7. Each layer provides a service to the layer above and uses the services below it. Corresponding layers follow agreed protocol rules, but the actual bits travel through the physical medium.

Functions at the sending computer

7. Application: provides network services used by an application, such as a web request, mail transfer or file transfer. For example, a browser creates an HTTP request for a document. The application layer concerns the network protocol, rather than every function of the browser itself.

6. Presentation: deals with the representation of information, including character encoding, serialization, compression and encryption. A text file may use UTF-8. In a practical Internet stack, libraries and protocols such as TLS perform some of these functions; there need not be a separate presentation-layer program.

5. Session: manages the establishment, coordination and ending of a logical dialogue. It can support checkpoints or recovery of a long exchange. Internet applications often implement session functions themselves, so an application login session is not identical to a TCP connection.

4. Transport: provides process-to-process delivery using port numbers. With TCP, the byte stream is divided into segments; sequence numbers, acknowledgments and retransmissions support reliable, ordered delivery. UDP instead sends separate datagrams without those reliability mechanisms.

3. Network: adds logical source and destination addresses and enables delivery across networks. IP packets are forwarded by routers according to the destination IP address and a routing table. A packet may pass through several routers before reaching the destination network.

2. Data link: packages a network-layer packet into a frame for the current link. Ethernet uses source and destination MAC addresses and a frame check sequence (FCS). A switch normally forwards frames using destination MAC addresses within the LAN.

1. Physical: converts frame bits into electrical, optical or radio signals. It specifies matters such as signaling, connectors, modulation and bit timing. The medium carries signals; it does not understand filenames, ports or IP routing.

Encapsulation: adding control information

An upper-layer data unit becomes the payload of the next lower layer. Headers identify addresses, protocol types or control fields; a link-layer trailer may carry an error-detection value. This wrapping process is called encapsulation.

```
Application data
[ TCP header | application data ]
[ IP header | TCP header | application data ]
[ MAC header | IP | TCP | data | FCS ]  ->  bits on the link
```

Example: retrieving a file using HTTP over TCP

Assume the server address is already known and use ordinary HTTP for a simple example. The browser sends an HTTP request over a TCP connection. TCP adds port and sequence information; IP adds the client and server IP addresses; Ethernet adds the MAC addresses of the client and its next hop. Signals carry the resulting frame onto the link.

If the server is on another network, the first Ethernet destination is the default gateway's MAC address. A router removes the incoming link frame, examines the IP packet and constructs a new frame for the outgoing link. Thus link addresses change from hop to hop; the end-host IP addresses normally remain the same when there is no NAT.

Decapsulation at the receiving computer

The physical interface recovers bits. Ethernet checks the frame and removes its link header and trailer. IP checks and removes its header and selects the upper-layer protocol. TCP verifies its checksum, orders the bytes, removes duplicates and delivers the stream to the correct application socket. The application interprets the HTTP message and saves or displays the requested data.

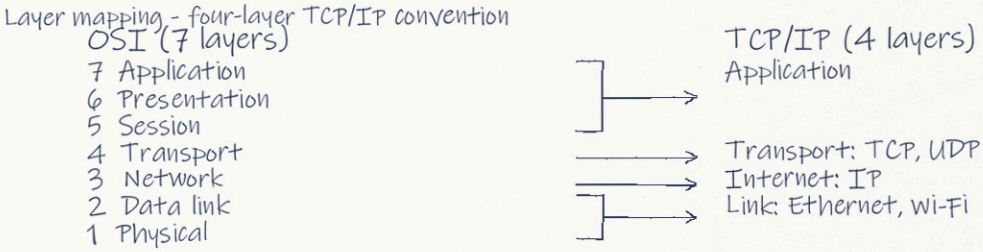
Encapsulation therefore adds information while moving down the sender's stack; decapsulation interprets and removes it while moving up the receiver's stack. Peer layers communicate logically using protocols, while adjacent layers interact through local service interfaces.

Tutorial sheet 1

Q2. OSI and TCP/IP reference models

OSI is a seven-layer reference model for describing network functions. TCP/IP is the architecture associated with the Internet protocol suite and is commonly shown using four layers. Many teaching diagrams split its link layer into physical and data-link layers to obtain a five-layer Internet model. Both presentations are valid when their convention is stated.

Layer mapping - four-layer TCP/IP convention



The mapping is approximate: real protocols do not always fit exactly one OSI layer. For example, TLS provides encryption and representation-related services above transport, while an application may maintain its own session state.

What the layers do

In OSI, application provides network services, presentation represents data, and session manages dialogues. Transport serves processes, network handles logical addressing and routing, data link serves one link, and physical transmits signals. The model separates these responsibilities explicitly.

In TCP/IP, application includes protocols such as HTTP, DNS and SMTP, together with the session and representation functions needed by applications. Transport includes TCP and UDP. The Internet layer contains IP and related control mechanisms such as ICMP. The link layer covers local-network access, framing and the technology used to move packets over a link.

Similarities

1. Both divide communication into layers, making a complex system easier to understand, implement and troubleshoot. A higher layer can use a lower-layer service without knowing every detail of its implementation.

2. Both distinguish application functions from transport, and transport from routing across networks. Both support encapsulation at the sender and reverse processing at the receiver.

3. Both use protocols between peer entities, interfaces between neighboring layers and addressing appropriate to different scopes. A port identifies an application endpoint, an IP address identifies a network-layer interface, and a MAC address identifies an interface on a local link.

4. Both allow different underlying media. An Internet application can run over Ethernet, Wi-Fi or other links as long as the necessary service is available.

Main differences

Layer count: OSI has seven named layers; the conventional TCP/IP model has four. OSI gives session and presentation their own layers, whereas TCP/IP puts these functions within its broader application layer.

Purpose: OSI provides a general reference architecture and terminology. TCP/IP describes a widely implemented collection of interoperating protocols. The OSI protocol suite also existed, but the reference model and the OSI protocol suite are different things.

Development: TCP/IP grew with working research networks and their practical protocols. OSI emphasized a general layered framework and formal service definitions. This difference helps explain why a practical TCP/IP stack does not perfectly match all OSI boundaries.

Network service: OSI's architecture can describe connection-oriented and connectionless network services. The Internet layer uses connectionless, best-effort IP delivery. Reliability, when required, can be supplied by an end-to-end transport such as TCP.

Layer boundaries: OSI describes service, interface and protocol distinctions more explicitly. TCP/IP has broader layers and does not impose separate session and presentation interfaces on every implementation.

Advantages and limitations

OSI advantages: precise vocabulary, clear division of responsibilities, independent layer design, and a useful framework for teaching and fault isolation. Its limitations are abstraction, some overlapping functions, and imperfect correspondence with deployed protocol stacks. Merely following the diagram does not guarantee interoperability.

TCP/IP advantages: practical interoperability, an open protocol ecosystem, scalable IP routing and support for heterogeneous networks. TCP and UDP allow applications to choose different transport behavior. Its limitations include less separation of upper-layer functions and a link layer that groups several different responsibilities.

Neither model alone guarantees security, low delay or reliable delivery. IP is best effort; security and reliability depend on the selected protocols and configuration. For example, TCP supplies reliable ordered bytes, while TLS adds authentication and encryption above TCP.

For design and troubleshooting, OSI is a useful explanatory framework. For identifying the protocols that actually exchange packets on the Internet, TCP/IP is the more direct description. They are complementary views of the same communication process.

Tutorial sheet 1

Q3. Systematic troubleshooting using all seven layers

Begin by defining the symptom: which users, devices, applications and destinations are affected, and when did the fault start? Record recent changes and compare with a working device. Test one hypothesis at a time, preserve useful logs and retest after each correction. A bottom-up approach is effective for general connectivity failures; a clearly application-specific symptom may justify starting higher.

Layer 1 - Physical

Check power, cable seating, link LEDs, damaged connectors, switch port status and optical transceiver compatibility. On wireless links, inspect signal strength, distance and interference. A disconnected cable or failed port prevents higher layers from working, even if IP settings are correct.

Example: a desktop reports that its network cable is unplugged. Try a known-good cable and port, then confirm that link is restored. If errors persist on a fiber link, inspect optical levels and connector cleanliness. Do not repeatedly change application settings for a missing physical link.

Layer 2 - Data link

Check Ethernet errors, speed and duplex agreement, VLAN membership, the switch MAC-address table and local neighbor resolution. Verify that the endpoint is on the intended VLAN. A wrong VLAN can produce a working link light while preventing communication with the intended gateway.

Example: a PC has the correct IP configuration but cannot resolve its gateway's MAC address. Inspect ARP entries, VLAN assignment and local switching. A rising FCS-error counter suggests corrupted frames; a duplex mismatch can cause severe loss. Correct the actual link or switching fault and retest.

Layer 3 - Network

Check the IP address, prefix or subnet mask, default gateway and routing table. Look for duplicate addresses and an unintended automatic address. Test the local address, a same-subnet host, the gateway and then a remote IP. Use ping and traceroute as evidence, remembering that ICMP may be filtered.

Example: local printers work but remote servers do not. The gateway may be missing, a route may be absent, or an IP firewall rule may block traffic. Verify the next hop and return route. Successful loopback only tests the local stack; it does not establish that the cable or router is working.

Layer 4 - Transport

Check the required TCP or UDP port, whether the service is listening, and whether firewalls allow the conversation. Inspect socket state and packet captures. For TCP, repeated SYNs with no reply may indicate a blocked path or unavailable host; an RST often indicates rejection or no listener.

Example: a server answers ping, but TCP port 443 cannot be reached. Check the web-service listener and firewall policy. ICMP reachability does not prove that a TCP service is usable. Also examine retransmissions, connection resets and transport timeouts when performance is intermittent.

Layer 5 - Session

Check whether the application can create and maintain its dialogue. Examine session expiry, reconnect behavior, login state and any session-service logs. Real TCP/IP systems often implement these functions inside applications, so the OSI label is a diagnostic viewpoint rather than a separate operating-system component.

Example: a user can open a service but is repeatedly logged out after a short interval. Investigate expired session tokens, inconsistent server-side session state or incorrect time settings. A successful TCP connection alone does not prove that an application session remains valid.

Layer 6 - Presentation

Check data format, character encoding, compression compatibility and encryption negotiation. Inspect TLS certificate validity, hostname matching and compatible protocol settings where encryption is used. An error here may leave network and transport connectivity fully operational.

Example: TCP connects to a website, but the client reports a certificate error. Check the certificate chain, expected hostname and clock. Another example is unreadable text caused by inconsistent character encoding. Repair the representation or trust configuration instead of changing IP addresses.

Layer 7 - Application

Verify the application configuration, DNS resolution, URL, credentials, permissions and service logs. Use an appropriate application-level test rather than relying only on ping. DNS is an application-layer service, even though its failures often appear to users as general network failures.

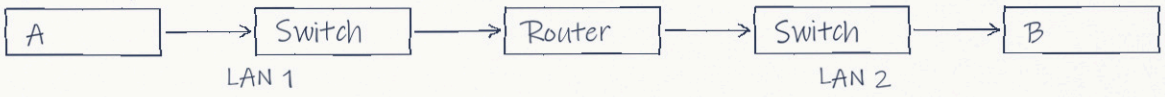
Example: a hostname fails to resolve while direct IP connectivity works. Test the configured DNS server and requested record. If the HTTP server responds with 404 or 403, connectivity exists; investigate the requested resource or access policy.

Finally, repeat the original user action, test another device or destination, and document the cause and fix. A lower-layer success narrows the search but does not guarantee that every higher-layer service works. Faults can also involve several layers, such as MTU problems appearing as stalled application transfers.

Tutorial sheet 1

Q4. File transfer between different networks

Assume an Ethernet LAN at each end, ordinary IP routing, no NAT and a TCP-based file-transfer application. Let A = 192.168.1.10/24, its gateway = 192.168.1.1, and B = 192.168.2.20/24. A simple route is shown below; a real path may contain more routers.



1. Application, presentation and session: the sending program opens the file, establishes the required application dialogue and encodes the data. If encryption is used, it is applied by the relevant application/security protocol. If B is identified by a name, DNS resolves the name before the main transfer.

2. Transport: TCP first establishes a connection, then numbers the file's bytes and sends them in segments. Source and destination ports identify the communicating applications. ACKs, checksums and retransmissions support ordered, reliable delivery; flow and congestion control limit outstanding traffic.

3. Network: IP adds A's source address and B's destination address. A compares B's address with its own subnet prefix and sees that B is remote. It therefore chooses its default gateway instead of trying to send directly to B on the local Ethernet.

4. Data link and physical: A uses ARP to learn the gateway's MAC address if necessary. Its first frame has source MAC = A and destination MAC = gateway, while its IP packet still has source IP = A and destination IP = B. The interface sends the frame as signals.

5. Switching and routing: a LAN switch forwards the frame toward the gateway using its MAC table. The router removes the frame, consults the route for B, reduces the IP TTL, and sends the packet in a new link frame. On the last Ethernet, ARP resolves B's MAC address. The final frame uses the router's outgoing MAC as source and B's MAC as destination.

6. Reception: B checks and removes the link framing, accepts the IP packet and passes its payload to TCP. TCP reorders received bytes, discards duplicates, requests recovery through its ACK behavior and delivers the stream to the receiving application. The program reconstructs the file and performs its own completion checks.

ACKs travel back toward A using the reverse communication process, although their network route need not be identical. Ordinary switches do not select routes by the final IP destination. MAC addresses apply to each local hop; IP addresses normally identify the original endpoints across the routed path. NAT is a separate case that can change those IP addresses and ports.

Q5. Why use a layered architecture?

A layer groups related networking responsibilities and offers a defined service to the next layer. A protocol specifies how corresponding entities communicate; an interface specifies how adjacent layers interact locally. This distinction lets an application use transport delivery without implementing Ethernet signaling or router forwarding.

Modularity: a large communication problem is divided into manageable functions. Designers can work on addressing, error handling or application semantics separately, provided the agreed service boundaries are preserved.

Interoperability: common protocol formats let equipment and software from different vendors communicate. For example, a TCP application can exchange data even when one end uses Wi-Fi and the other uses Ethernet.

Independent evolution: a link can be upgraded from copper to fiber without rewriting the application's file format. The new lower layer must still supply the service expected above it. This reduces the effects of implementation changes.

Reuse and reduced complexity: many applications share TCP/IP, and IP can run over many link types. Developers reuse established functions rather than designing a complete network for every application. Layers also support clearer documentation and focused conformance testing.

Fault isolation: the same structure supports systematic diagnosis. A broken cable belongs to a different responsibility from an incorrect route or an invalid application password, so tests can progressively narrow the cause.

Limits: headers, processing and data copies create overhead; error recovery may be repeated at different layers. Strict separation can hide useful information, and some functions such as security span several layers. Layering is a design aid whose interfaces need careful definition, rather than a guarantee of optimum performance in every situation.

Tutorial sheet 1

Q6. TCP: establishment, reliability and control

Transmission Control Protocol is a connection-oriented, full-duplex transport protocol. It gives applications a reliable, ordered byte stream across an underlying best-effort IP network. A connection is identified by the pair of endpoint IP addresses and port numbers. TCP does not preserve application message boundaries: the receiver may read a different-sized chunk from the one the sender wrote.

Connection establishment

The active opener sends SYN with an initial sequence number x. The listener replies with SYN + ACK, its own initial sequence number y and acknowledgment x + 1. The initiator returns ACK for y + 1. Both ends synchronize sequence spaces and negotiate applicable options, such as maximum segment size and window scaling. A SYN consumes one sequence number.

The handshake creates state at the endpoints; it does not reserve a fixed route through the network. Packets belonging to the same connection can still follow different IP routes. The connection must also remain within application and operating-system resource limits.

Reliable data transfer

TCP numbers bytes, so a segment's sequence number normally identifies its first data byte. If a segment starts at 1001 and carries 500 bytes, it contains bytes 1001 through 1500; an ACK of 1501 means that all preceding bytes have been received in order.

The checksum helps detect corruption. The receiver can buffer out-of-order data and uses cumulative ACKs to report the next expected byte. The sender retains unacknowledged bytes and retransmits after a timeout or another loss indication. Duplicate data is recognized through sequence numbers and is not delivered twice to the application.

Example: bytes 1001-1500 are lost but 1501-2000 arrive. The cumulative ACK remains 1001 because the gap is unresolved. After the missing bytes arrive, the ACK can advance to 2001. If Selective Acknowledgment is negotiated, the receiver can also report received blocks beyond the gap, helping the sender choose what to retransmit.

The retransmission timeout is based on measured round-trip behavior and its variation. Timeouts back off when repeated losses occur. TCP can recover from many transient losses, but it may eventually report a connection failure; reliability does not mean guaranteed success on a permanently failed path.

Flow control - protect the receiving endpoint

The receiver advertises a receive window, rwnd, based on how much more data it can accept. The sender limits outstanding bytes so that it does not overrun this advertised capacity. As the receiving application consumes data, the receiver can advertise more space.

If rwnd becomes zero, normal new-data transmission pauses. TCP uses window probes so that a lost window-update message does not cause both ends to wait forever. This is different from retransmission caused by network congestion.

Congestion control - protect the network path

The sender also maintains a congestion window, cwnd. Slow start increases the sending allowance rapidly at first; congestion avoidance increases it more cautiously. Loss or other congestion indications cause the sender to reduce its rate. Classical Reno uses duplicate ACKs for fast retransmit and fast recovery.

Outstanding data must fit within both limits: allowed flight size is at most min(rwnd, cwnd). If rwnd = 32 KB and cwnd = 12 KB, the network limit is tighter. If rwnd falls to 4 KB, the receiver limit becomes tighter. The amount of new data available to send also subtracts bytes already outstanding.

Connection termination

Each direction closes independently. In a typical orderly close, A sends FIN; B ACKs it. B later sends its own FIN and A acknowledges it. The ACK and FIN can sometimes be combined, so four separate segments are common rather than mandatory. A FIN, like a SYN, consumes one sequence number.

An endpoint can continue receiving after closing its sending direction: this is a half-close. The active closer normally enters TIME-WAIT for twice the maximum segment lifetime, allowing delayed duplicates to expire and the final ACK to be repeated if needed. RST provides an abrupt abort rather than an orderly end of the byte stream.

TCP reliability, flow control and congestion control solve different problems. Together they make TCP suitable for file transfer, email and many web exchanges where missing or reordered bytes must be repaired before the application processes the stream.

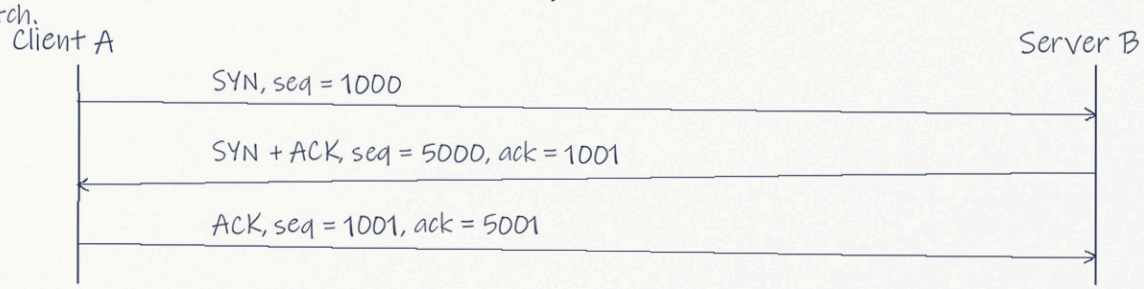
Tutorial sheet 1

Q7. TCP three-way handshake

A TCP connection has two independently numbered byte streams, one in each direction. Before normal data exchange, both endpoints must learn the other's initial sequence number and confirm that their own initial number was received. The three-way handshake performs this synchronization and reduces confusion caused by delayed segments from old connection attempts.

Simple example

Let the client choose initial sequence number 1000 and the server choose 5000. These are illustrative values, not fixed numbers used by TCP. The server is initially listening for connections at its service port. Time runs downward in the sketch.



Step 1 - client sends SYN

The client sends a segment with SYN = 1 and sequence number 1000, then enters SYN-SENT. It is asking to establish a connection and announcing the starting point of its sequence space. In the ordinary initial SYN, ACK is not set, so its acknowledgment field is not a valid acknowledgment of server data.

Step 2 - server sends SYN + ACK

The server receives the request and replies with SYN = 1, ACK = 1, sequence number 5000 and acknowledgment number 1001. It enters SYN-RECEIVED. ACK 1001 confirms the client's SYN, because a SYN occupies one sequence number even without application bytes.

The server also announces its own initial sequence number. Combining this SYN and the acknowledgment of the client's SYN saves a separate message. At this stage, the server still needs confirmation that the client received the server's SYN.

Step 3 - client sends ACK

The client receives the SYN + ACK, acknowledges it with ACK = 5001 and uses sequence number 1001. It enters ESTABLISHED; the server enters ESTABLISHED after receiving this acknowledgment. A pure ACK consumes no sequence number, so the first client data can also begin at 1001.

For example, if the client next sends 200 bytes beginning at 1001, they are bytes 1001 through 1200. Once those bytes are received in order, the server acknowledges 1201. The server's own first application byte has sequence number 5001.

Why are three messages required?

The first message alone proves only that the server may receive a request. The second tells the client that its SYN reached the server and provides the server's starting sequence number. The third tells the server that its response reached the client and that the client accepts the server's sequence number.

With only two messages, the server cannot distinguish a completed exchange from a situation in which its SYN + ACK never reached the client. It could allocate an apparently established connection in response to an old delayed SYN even when no current client wanted that connection.

The final acknowledgment ties the exchange to the current sequence numbers. This helps avoid old-duplicate confusion, although the handshake is not cryptographic authentication and does not by itself provide encryption or protection against every attack.

Loss and option negotiation

If the initial SYN or SYN + ACK is lost, the appropriate endpoint retransmits after its timer expires. If the final ACK is lost, the server may repeat SYN + ACK, which the client acknowledges again; a valid acknowledgment accompanying later client data can also complete establishment. A normal pure ACK is not itself given a separate acknowledgment.

SYN segments can carry options such as MSS, window scaling and SACK permission. The MSS states the maximum TCP payload the sender of that option is willing to receive. Negotiating a connection does not promise a particular bandwidth and does not create a dedicated circuit through intermediate routers.

Thus the handshake synchronizes two sequence spaces and confirms communication in both directions. Subsequent checksums, ACKs, retransmissions and window mechanisms provide the ongoing reliable byte-stream service.

Tutorial sheet 1

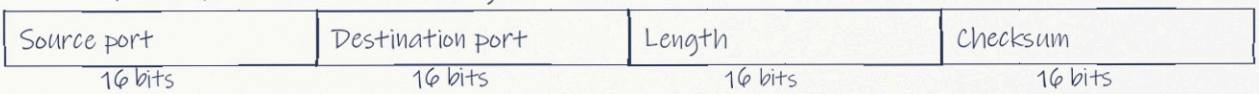
Q8. Detailed comparison of TCP and UDP

TCP and UDP are transport-layer protocols carried by IP. Both use source and destination ports to identify application endpoints, and both have checksums for detecting accidental corruption. Their main difference is the delivery service exposed to the application: TCP supplies an ordered byte stream, while UDP supplies individual datagrams.

Header structure and overhead

A TCP header is normally at least 20 bytes and can extend to 60 bytes with options. It contains source and destination ports, sequence and acknowledgment numbers, data offset, control flags, receive window, checksum and urgent pointer. Options may include MSS, timestamps, window scaling or selective acknowledgment information.

A UDP header is exactly 8 bytes: four fields of 16 bits each. Its length field includes both header and payload. UDP has no transport sequence number, acknowledgment number or receive-window field.



For a 100-byte application payload, assume IPV4 without options and no TCP options. TCP + IPV4 adds 20 + 20 = 40 bytes, producing 140 bytes before link framing. UDP + IPV4 adds 8 + 20 = 28 bytes, producing 128 bytes. The transport-only overhead is therefore 20% and 8% of the payload, respectively; real overhead also depends on ACKs, options and packetization.

Reliability, order and message boundaries

TCP acknowledges bytes, retransmits missing data, suppresses duplicates and delivers bytes in order. When data is lost, later bytes may have to wait before reaching the application. TCP does not preserve write boundaries, so applications need their own message framing, such as a length field or delimiter.

UDP does not promise delivery, ordering or duplicate suppression. A datagram may be lost, delayed, duplicated or received after a later one. UDP preserves datagram boundaries: the application sees messages rather than a continuous stream, subject to its receive buffer and API behavior.

The UDP checksum detects corruption but does not repair it. In IPV4 a UDP checksum can be omitted under the protocol rules; in IPV6 it is generally required, apart from narrowly defined exceptions. A checksum is not a guarantee of reliability and is not cryptographic protection.

Connection management

TCP normally uses a three-way handshake before ordinary exchange and keeps per-connection state. An orderly close uses FIN and ACK processing; RST aborts a connection. This creates setup cost and state-management work, but supports coordinated reliable communication.

UDP has no built-in handshake or termination exchange. An application can send a datagram without establishing a UDP protocol connection. An operating system's connected-UDP socket mainly selects a peer and changes local API behavior; it does not introduce a TCP-like network handshake.

Flow control and congestion control

TCP uses the receiver's advertised window to avoid overwhelming the receiving application. Its congestion-control algorithm also limits traffic according to the condition of the network path. Both controls influence how much data may be outstanding.

UDP contains neither built-in flow control nor built-in congestion control. An application or higher-level protocol must add appropriate control where needed. It is incorrect to assume that a UDP application should send at an unlimited rate merely because UDP itself does not regulate it.

Delay, resource cost and applications

TCP's retransmissions and in-order delivery favor correctness but can increase delay after loss. TCP also needs connection state, timers and retained unacknowledged data. UDP's simpler transport can suit short exchanges or applications that prefer a missing update over a late one. UDP is not automatically faster under every workload.

TCP examples: conventional HTTP/1.1 or HTTP/2 connections, SSH, SMTP and many file transfers. They generally benefit from reliable ordered bytes. UDP examples: many DNS queries, DHCP, real-time media and time-sensitive game updates. DNS can also use TCP, so an application name does not always determine one transport.

Higher-level protocols can build reliable delivery and security over UDP. QUIC is an example, and HTTP/3 uses QUIC rather than TCP. Therefore, UDP's own lack of reliability does not imply that every application using UDP is unreliable.

Choose TCP when the application naturally needs a reliable stream. Choose UDP when datagram semantics and application-controlled timing or recovery are useful, while still designing appropriate congestion response and security for the actual application.

Tutorial sheet 1

Q9. TCP congestion-control mechanisms

Congestion occurs when offered traffic exceeds the capacity of a link or router queue. Delay and queue occupancy rise, and packets may be dropped. TCP's sender controls its outstanding data using a congestion window, cwnd. The receive window, rwnd, protects the receiving endpoint instead; it is a separate limit.

Usable flight limit = min(cwnd, rwnd). The slow-start threshold, ssthresh, separates rapid startup growth from more cautious growth. MSS denotes the maximum segment payload used by the sender. The explanation below uses the classical Reno-style mechanisms; deployed TCP algorithms can use other growth rules.

1. Slow Start

At the beginning of an illustrative connection, let cwnd = 1 MSS and ssthresh = 8 MSS. The sender transmits only what its window permits. As ACKs for new data arrive, it increases cwnd by approximately one MSS per ACK, giving roughly exponential growth across round-trip times when each segment is acknowledged.

Idealized evolution: 1, 2, 4, 8 MSS over successive RTTs. This is called slow start because it begins conservatively, even though the subsequent growth is rapid. Real initial windows may be larger; 1 MSS here is a teaching assumption, not a universal requirement.

Delayed ACKs, application-limited traffic and the receive window can alter the simple doubling pattern. Growth also stops being useful if the application has no more data. Once the threshold is reached, the sender transitions to congestion avoidance under the chosen rule.

2. Congestion Avoidance

When cwnd is at or above the threshold, growth becomes approximately one MSS per RTT instead of doubling. One common per-ACK approximation is:

$$cwnd = cwnd + MSS \times MSS / cwnd$$

For cwnd = 8 MSS, each appropriate ACK adds about 1/8 MSS; roughly eight such ACKs give about one MSS of total increase. This additive increase probes for extra capacity cautiously. The idealized window then progresses 8, 9, 10, 11 MSS across RTTs.

3. Fast Retransmit

Suppose a segment is lost but later segments arrive. The receiver continues acknowledging the next byte it needs, producing duplicate ACKs. In the basic Reno rule, three duplicate ACKs trigger retransmission of the presumed missing segment without waiting for the retransmission timeout.

One duplicate ACK is not enough evidence: reordering can also produce duplicate ACKs. Three duplicate ACKs are a heuristic signal that later data is arriving but a gap remains. This speeds recovery when enough later segments are in flight; a loss near the end of a small transfer may still require a timeout.

4. Fast Recovery - classical Reno

After the third duplicate ACK, set ssthresh to approximately half the flight size, but at least 2 MSS. Retransmit the missing segment, and set cwnd to ssthresh + 3 MSS. The temporary addition reflects the segments indicated as having left the network by those duplicate ACKs.

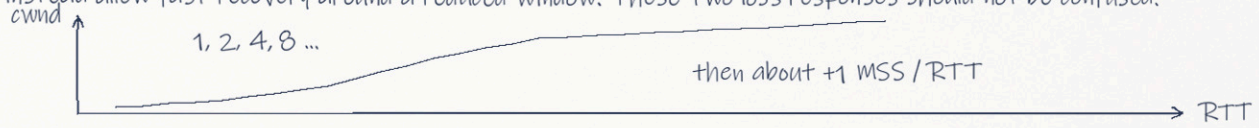
For each further duplicate ACK, temporarily increase cwnd by one MSS and send new data if the window allows. When the ACK covering the recovered outstanding data arrives in the simple single-loss case, reduce cwnd to ssthresh and continue in congestion avoidance.

Example: flight size is 16 MSS. Set ssthresh = 8 MSS and temporarily cwnd = 11 MSS. After successful recovery, cwnd returns to about 8 MSS, then grows cautiously. Basic Reno handles multiple losses in one window less effectively than refinements such as NewReno or SACK-based recovery.

Timeout response and overall behavior

A retransmission timeout is a stronger indication that useful feedback has stopped. In the classical rule, set ssthresh to max(flight size / 2, 2 MSS), reduce cwnd to 1 MSS, retransmit, and resume slow start. The retransmission timeout itself is backed off to avoid repeated aggressive retries.

If 16 MSS were outstanding, a timeout gives threshold 8 MSS but restarts at 1 MSS. Three duplicate ACKs instead allow fast recovery around a reduced window. These two loss responses should not be confused.



Together, rapid initial growth, additive increase and multiplicative reduction help TCP seek available capacity while responding to overload. Loss can also arise from wireless corruption, so loss is an imperfect congestion signal. Explicit Congestion Notification can provide a congestion indication without requiring packet loss when supported.

Tutorial sheet 1

Q10. Go-Back-N ARQ and Selective Repeat ARQ

ARQ means Automatic Repeat reQuest. Both protocols use sequence numbers, error detection, ACKs and retransmission to recover from lost or corrupted frames. Both permit a sender window containing several unacknowledged frames, so the sender can keep transmitting while earlier frames propagate through the link.

Go-Back-N (GBN)

The sender can have up to N frames outstanding. The conventional receiver accepts only the next expected frame and discards later out-of-order frames. Its receive window is therefore one frame. It sends cumulative acknowledgments indicating the highest in-order progress.

If the oldest outstanding frame times out, the sender retransmits it and all later frames already sent but not cumulatively acknowledged. A standard implementation uses one timer for the oldest outstanding frame. Losing one frame can therefore make the sender repeat several frames that had reached the receiver but were discarded.

Selective Repeat (SR)

The receiver accepts correct frames within its receive window even when an earlier frame is missing. It buffers out-of-order frames and acknowledges them individually or selectively. Data is delivered upward in order once the missing gap is filled.

The sender retransmits only the frames whose successful reception has not been acknowledged. It must track acknowledgment status separately and support a separate timeout decision for each outstanding frame, although implementations can use one timer service to manage many deadlines.

Direct comparison

Retransmission: GBN repeats the missing frame and subsequent outstanding frames; SR repeats only the missing or damaged frames that still require recovery.

Receiver buffering: conventional GBN has no need to retain out-of-order frames. SR needs memory and bookkeeping for correctly received frames waiting beyond a gap.

Acknowledgments: GBN uses cumulative in-order ACKs. SR uses individual or selective information. A lost cumulative ACK can be covered by a later ACK; SR may re-ACK a duplicate so that a lost selective ACK does not cause indefinite retries.

Sender complexity: GBN is relatively simple, with a base, next sequence number and oldest-frame timer. SR keeps more per-frame state. Both retain copies of data that may need retransmission.

Efficiency: on a clean link both can pipeline successfully. When errors occur and the window or propagation delay is large, SR usually saves bandwidth by keeping valid out-of-order data. Its extra receiver memory and control logic are the tradeoff.

Small example - window size 4, frame 2 lost

Use frame labels 1, 2, 3, 4, and let ACK k mean that frame k is acknowledged; GBN ACKs are cumulative in this example. ACK numbering conventions differ between books, so the meaning is stated explicitly.

Original sends:	1	2 (lost)	3	4
GBN repeats:		2	3	4
SR repeats:		2 only		

In GBN, frame 1 is accepted but 3 and 4 are discarded because 2 is missing. After timeout, resend 2, 3 and 4: total transmissions = 4 + 3 = 7. In SR, frames 3 and 4 are kept and acknowledged. Only 2 is resent: total = 4 + 1 = 5. Assume the retransmissions and ACKs are not lost.

Sequence-number space and window size

If the sequence number has m bits, there are 2^m different labels before wraparound. In conventional GBN, the sender window must be at most $2^m - 1$, with receive window 1. Leaving one label outside the sender window avoids confusing a new cycle with old unacknowledged data.

For standard SR with equal sender and receiver windows W, require $2W \le 2^m$, so $W \le 2^{(m-1)}$. The receive window must not overlap sequence labels that could still represent old duplicates. Otherwise, an old retransmitted frame could be mistaken for new data after the window advances.

Example: with m = 3, labels are 0 through 7. GBN allows a sender window of at most 7; SR allows equal windows of at most 4. To use an SR window of 5, at least 4 sequence bits are required because $2 \times 5 = 10$ exceeds 8.

Both schemes detect corruption and recover through feedback and retries; neither reconstructs arbitrary damaged bits by itself. GBN suits simpler receivers and low-error links. SR is attractive when retransmitting already received data would be expensive, provided buffering and more detailed control are available.

Computer Networks - Tutorial sheet 2

Q1. Connecting factory buildings 2 km apart

Recommendation: use an optical-fiber link, preferably a suitably rated single-mode fiber installation with compatible optical Ethernet transceivers. Select the fiber, wavelength and optics for the required speed and a supported reach exceeding the actual cable route. The link budget must include fiber, connectors, splices and an engineering margin.

Attenuation: fiber has much lower loss over this distance than a normal copper Ethernet channel. Ordinary twisted-pair Ethernet is generally designed around a 100 m channel for common variants; a 2 km direct run is far beyond that example. Single-mode fiber is appropriate for kilometer-scale links and avoids the modal-dispersion limits of many multimode options.

Noise immunity: light in an all-dielectric fiber is unaffected by electromagnetic interference from motors, welding equipment or power cables. Fiber also provides electrical isolation between buildings, avoiding a conductive data path between their different ground potentials. Any metallic armor or support hardware still needs installation practices appropriate to it.

Bandwidth: suitable fiber and optics support high data rates and allow capacity upgrades. The actual rate is limited by the transceivers, switches, distance rating and optical budget, rather than fiber automatically providing unlimited bandwidth.

Cost: optical transceivers, termination, splicing and civil work can make initial installation more expensive. However, long reach and noise immunity can reduce the need for intermediate powered repeaters and improve reliability in a factory environment. Reuse of existing ducts can substantially affect project cost.

Simple layout: building A switch -- fiber pair -- building B switch. A matching bidirectional optical pair can also operate over one fiber if designed for that purpose. For an ordinary duplex link, use one fiber for each direction and protect the route mechanically.

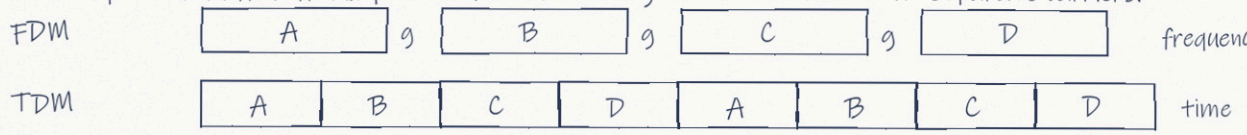
A wireless bridge is an alternative where cabling is impractical, but requires suitable line of sight, link planning and consideration of interference. Given the stated attenuation and EMI problems, fiber is the strongest direct recommendation.

Q2. Carrying four independent channels

Multiplexing combines several logical channels on one physical link. A multiplexer combines them at the sender and a demultiplexer separates them at the receiver. Four channels do not each obtain the full link capacity for free; the aggregate physical capacity must cover their combined rates plus the required overhead.

Option A - frequency-division multiplexing (FDM)

Assign each channel a separate carrier-frequency band. All four can then transmit at the same time in different portions of the available spectrum. Receiver filters select the desired band and demodulate its content. FDM is a natural explanation when the independent voice or data signals are modulated onto separate carriers.



Guard bands are unused frequency gaps between neighboring channels. They allow for practical filters and imperfect spectral confinement, reducing interference between adjacent channels. If each channel needs bandwidth B and every internal guard band is g, an idealized four-channel allocation needs at least $4B + 3g$, excluding any extra outer-edge allowance.

For $B = 4$ kHz and $g = 1$ kHz, the allocation is $4 \times 4 + 3 \times 1 = 19$ kHz. Guard bands improve separation but consume spectrum that cannot carry useful channel payload. Proper filtering and carrier stability are still necessary.

Option B - time-division multiplexing (TDM)

For a digital link, sample or digitize voice as needed and assign channels repeating time slots: A, B, C, D, then repeat. Each source uses the full channel during its allocated slot. Although only one sends in a given slot, all four receive continuing service over successive frames.

Slots prevent overlap when the multiplexer and receiver share correct frame and bit timing. Guard times may be inserted where timing uncertainty requires them; these are time gaps, unlike FDM guard bands. Framing bits and synchronization consume part of the link capacity.

If each source supplies 64 kbit/s, the payload link rate must be at least $4 \times 64 = 256$ kbit/s, with extra capacity for framing or other overhead. Fixed synchronous slots are simple but waste capacity when a source is idle; statistical TDM can allocate service dynamically but needs identifiers and buffering.

Either scheme meets the requirement with adequate resources. Choose FDM when simultaneous separate frequency bands are suitable, or TDM for a digital link with coordinated time slots. The essential separation is frequency plus guard bands in FDM, or time plus synchronization in TDM.

Tutorial sheet 2

Q3. NRZ, NRZI and Manchester encoding

Given data bits: 1 0 1 1 0 0 1. A line code maps these bits to signal levels or transitions over fixed bit intervals. The convention must be stated because NRZI and Manchester are described with opposite polarity conventions in different references. H and L below mean the high and low signal levels.

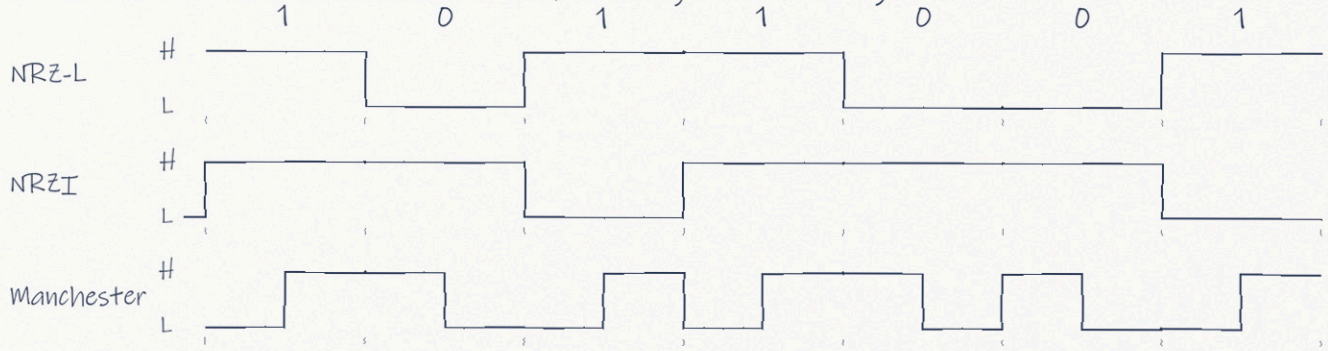
(a) Polar NRZ-L

Choose 1 = H and 0 = L. The level stays constant throughout each bit interval and does not return to a special zero level between bits. Therefore the seven transmitted levels are:

bit: 1 0 1 1 0 0 1

NRZ-L: H L H H L L H

There is a transition only when adjacent data bits differ. Bits 3 and 4 are both high; bits 5 and 6 are both low. The horizontal axis in each sketch is time, increasing from left to right.



Each pair of small ticks encloses one bit interval.

(b) NRZI - transition on a 1

Choose the initial level immediately before bit 1 as L. A 1 reverses the level at the start of its bit interval; a 0 leaves the previous level unchanged. This rule produces levels H, H, L, H, H, H, L for the seven intervals.

Step check: bit 1 is 1, so L changes to H. Bit 2 is 0, so stay H. Bit 3 is 1, so change to L. Bit 4 is 1, so change to H. Bits 5 and 6 are 0, so remain H. Bit 7 is 1, so change to L.

NRZI represents a bit by whether a transition occurs, rather than by one absolute voltage polarity. This can make decoding insensitive to reversing the overall signal polarity. However, with this transition-on-1 convention, a long run of zeros produces no transitions and makes clock recovery difficult. Some systems use the opposite transition rule, so the stated convention matters.

(c) Manchester - a mid-bit transition every time

Choose the IEEE 802.3 convention: 1 = L then H; 0 = H then L. Each bit interval is divided into two halves, and a transition at its midpoint identifies the bit. For the given sequence, the half-bit pairs are:

LH | HL | LH | LH | HL | HL | LH

The midpoint always contains a transition. At the boundary between bits, an extra transition may or may not occur, depending on the neighboring symbols. The waveform should therefore be decoded using the known bit centers, rather than treating every edge as a separate data bit.

Advantages and limitations

NRZ-L is straightforward and can use less signaling bandwidth than Manchester at the same data rate. Its weakness is poor clock information during long runs of identical bits; unbalanced patterns can also introduce a DC component or baseline wander.

NRZI encodes changes relative to the previous state, which helps with polarity inversion, but does not automatically eliminate transition-free runs. Bit stuffing, scrambling or another coding rule may be needed to ensure enough transitions for synchronization.

Manchester is self-clocking because every bit has a midpoint transition. With ideal equal positive and negative levels, each bit is balanced. Its cost is two signal elements per data bit and a greater signaling-bandwidth requirement than a comparable NRZ code.

For a data rate R bits/s, the bit duration is T = 1/R. Manchester changes its half-bit symbol every T/2, corresponding to a half-bit signaling clock of 2R. The sketches use simple high/low levels and show logical timing only, without analog edge shape or channel distortion.

Tutorial sheet 2

Q4. Pure ALOHA and Slotted ALOHA

In ALOHA, devices share a channel and transmit without reserving a dedicated path. If overlapping transmissions cause a collision, the sender must try again after a random delay. Assume equal-duration packets of time T, independent random transmission attempts with the standard Poisson offered-load model, and a collision channel where overlapping packets cannot both be decoded.

Pure ALOHA: a device transmits whenever its packet is ready. A packet is vulnerable to another start from T before its own start until T afterward, giving a vulnerable period of 2T. Even a partial overlap can destroy the packet under this model. The protocol is simple and does not require global slot synchronization.

Slotted ALOHA: time is divided into slots of duration T, and a device may start only at a slot boundary. Collisions then occur among devices choosing the same slot. The vulnerable period is T, because packets starting in neighboring correctly aligned slots do not overlap. Devices must share slot timing.

Throughput calculation

Let G be the mean number of attempts, including retransmissions, per packet time. Let S be the mean successful packets per packet time.

Pure ALOHA: $S = G e^{(-2G)}$

Slotted ALOHA: $S = G e^{(-G)}$

For pure ALOHA, $dS/dG = e^{(-2G)}(1 - 2G)$, so the maximum occurs at $G = 1/2$. Thus $S_{max} = 1/(2e) = 0.184$, or about 18.4% of the raw packet-time capacity.

For slotted ALOHA, $dS/dG = e^{(-G)}(1 - G)$, so the maximum occurs at $G = 1$. Thus $S_{max} = 1/e = 0.368$, or about 36.8%. These are idealized maximum normalized throughputs, not the success probability of every packet at every load.

Recommendation: use Slotted ALOHA if the sensor devices can maintain sufficiently accurate common slot timing. Under the stated traffic and collision assumptions, its peak throughput is twice that of pure ALOHA. If maintaining synchronization is too costly in battery power or timing overhead, the practical advantage may shrink.

The key assumption behind the recommendation is synchronized equal-length slots, with transmissions restricted to their boundaries. The numerical peaks also rely on the Poisson attempt model and no capture or multi-packet reception. Random backoff is still needed because slots reduce, rather than eliminate, collisions.

Q5. Shared Ethernet uses CD; Wi-Fi uses CA

CSMA means carrier-sense multiple access: a device checks whether the medium appears busy before transmitting. Two devices can still sense an idle medium and transmit nearly together, because signals take time to propagate and because a local observation may not reflect every other transmitter.

Traditional shared Ethernet: CSMA/CD

On a half-duplex shared Ethernet, a station can monitor the wired medium while transmitting. If it detects a collision, it stops the attempted frame, sends the required collision indication and retries after binary exponential backoff. Detecting a collision early avoids continuing to transmit a complete frame that has already been damaged.

Example: several computers connected through an old Ethernet hub share one collision domain. Simultaneous transmissions contend for the same medium, so CSMA/CD is appropriate. Full-duplex switched Ethernet has separate send and receive operation on each link and does not use CSMA/CD; this distinction is essential.

Wi-Fi: CSMA/CA

A wireless transmitter's own signal is much stronger at its radio than a remote station's signal, making reliable collision detection during transmission impractical in ordinary Wi-Fi. Hidden terminals can also be outside one another's sensing range while both reaching the access point.

Wi-Fi therefore tries to avoid collisions using carrier sensing, an interframe wait and random backoff. A backoff counter decreases while the channel is idle and pauses when it becomes busy. The station transmits when the counter reaches zero. A missing unicast ACK causes a retry after the applicable timeout and contention procedure.

Optional RTS/CTS exchanges and virtual carrier sensing can reduce hidden-terminal collisions. They add overhead and do not guarantee that collisions disappear. Broadcast and multicast handling also differs from the ordinary acknowledged unicast example.

Example: laptops in a classroom send through one access point, while some are too far apart to hear each other. CSMA/CA coordinates access and uses ACK feedback rather than trying to detect a collision while transmitting. CD reacts after detecting overlap; CA reduces its likelihood before transmission and infers failed delivery afterward.

Tutorial sheet 2

Q6. CRC for dataword 1101011011, generator 10011

The generator has 5 bits, so its polynomial degree is r = 4. Its polynomial is g(x) = x^4 + x + 1. The CRC remainder must therefore contain 4 bits, and four zero bits are appended to the dataword before division.

Dataword D = 1101011011

Augmented data = D × x^4 = 11010110110000

Perform modulo-2 division by 10011. Subtraction is XOR, with no borrow and no carry. At every step align the leading 1 of the divisor with the current leading 1 of the working dividend. Leading zeros are retained below so the alignment is easy to check.

Working value	XOR aligned divisor	Result
1 1 0 1 0 1 1 0 1 1 0 0 0 0	1 0 0 1 1 0 0 0 0 0 0 0 0 0	0 1 0 0 1 1 1 0 1 1 0 0 0 0
0 1 0 0 1 1 1 0 1 1 0 0 0 0	0 1 0 0 1 1 0 0 0 0 0 0 0 0	0 0 0 0 0 0 1 0 1 1 0 0 0 0
0 0 0 0 0 0 1 0 1 1 0 0 0 0	0 0 0 0 0 0 1 0 0 1 1 0 0 0	0 0 0 0 0 0 0 0 1 0 1 0 0 0
0 0 0 0 0 0 0 0 1 0 1 0 0 0	0 0 0 0 0 0 0 0 1 0 0 1 1 0	0 0 0 0 0 0 0 0 0 0 1 1 1 0

The final four bits are 1110.

After the final XOR, the remaining nonzero part is 1110, whose degree is less than 4. Therefore the required remainder is R = 1110. Replace the appended zeros with this remainder:

Transmitted codeword C = 1101011011 1110

C = 11010110111110 (14 bits)

Receiver's error check

The receiver divides the entire received 14-bit codeword by the same generator 10011. It does not append another four zeros. The correct codeword gives remainder 0000, so it passes the CRC check.

If the received polynomial is C'(x) = C(x) + E(x), then its remainder is the remainder of the error polynomial E(x), because C(x) is already divisible by g(x). A nonzero remainder indicates a detected error, so the receiver rejects the frame or requests retransmission according to the surrounding protocol.

For a concrete check, flip only the last transmitted bit: received word = 11010110111111. This differs by the error polynomial 1 and gives remainder 0001, so the error is detected. The CRC identifies corruption but does not determine the original message by itself.

A zero remainder means no error was detected, rather than proof that no corruption occurred. Error patterns divisible by the generator can be undetected. A degree-4 CRC detects every burst error of length at most 4 under the usual CRC construction; stronger guarantees depend on the generator and frame length.

Q7. Stop-and-wait utilization

Given one-way propagation delay Tp = 20 ms, frame transmission time Tt = 5 ms, and negligible ACK transmission time. Assume no errors, immediate ACK generation, negligible processing/queueing delay, and that the sender waits for the ACK before sending the next data frame.

One complete cycle

The sender transmits the frame from t = 0 to 5 ms. Its last bit reaches the receiver at 5 + 20 = 25 ms. The receiver immediately sends an ACK, which takes another 20 ms to propagate back. The sender can begin its next frame at about t = 45 ms.

Cycle time = Tt + 2Tp + Tack

= 5 + 2 × 20 + 0 = 45 ms

Utilization U = useful transmission time / cycle time

= 5/45 = 1/9 = 0.1111

Approximate Stop-and-Wait utilization = 11.1%

The sender uses the forward link to transmit data for 5 ms and waits for about 40 ms in each cycle. This utilization is the fraction of time spent sending data frames; if headers are counted separately from payload, payload efficiency is lower.

An equivalent formula is U = 1/(1 + 2a), where a = Tp/Tt. Here a = 20/5 = 4, so U = 1/(1 + 8) = 1/9. Using only 5 + 20 in the denominator would incorrectly omit the ACK's return propagation.

Effect of increasing propagation delay

For fixed Tt, U = Tt/(Tt + 2Tp) decreases as Tp increases. If the one-way delay doubles to 40 ms, U = 5/(5 + 80) = 5/85 = 5.88%. Stop-and-Wait becomes inefficient on links with a large bandwidth-delay product.

A sliding-window protocol can place more frames in flight. Under the same ideal timing, roughly W >= 1 + 2a = 9 frames are needed to keep transmission continuous. This comparison assumes no additional delay, ACK bottleneck or errors; it explains why pipelining improves long-delay link performance.

Tutorial sheet 2

Q8. 15 packets, window size 5; every fourth original lost

Label the packets P1 through P15. Interpret an original transmission as the first attempt to send a particular packet. The lost first attempts are therefore P4, P8 and P12. Retransmissions are not counted as new original attempts and are assumed successful. ACKs are never lost, as specified.

Timing and counting assumptions

Use conventional GBN with receiver window 1, cumulative ACKs, one oldest-frame timeout and no fast retransmit. For SR, use individual/selective ACKs and receiver buffering for a window of 5. Count data-packet transmissions only; do not count ACKs as data transmissions.

For the GBN trace below, ACKs for in-order packets arrive soon enough to let the sender fill the available window before the oldest missing packet times out. At a timeout, retransmit the complete outstanding group before sending new packets. Each retransmitted group succeeds and is acknowledged before the next new-data phase. These assumptions make the total reproducible.

Go-Back-N: explicit transmission trace

Phase 1 - original sends P1 through P8: P1, P2 and P3 are accepted, and their ACKs slide the window. The first P4 is lost, so the receiver continues waiting for P4. P5, P6 and P7 arrive out of order and are discarded. The first P8 is also lost. The outstanding window is now P4-P8, containing 5 packets.

Phase 2 - timeout of P4: resend P4, P5, P6, P7 and P8. All 5 retransmissions succeed in order. The cumulative acknowledgment advances through P8. P8 is successful here because only its original attempt was designated as lost.

Phase 3 - original sends P9 through P15: P9, P10 and P11 are accepted and acknowledged. The first P12 is lost. P13, P14 and P15 are discarded because P12 is missing. There are no more original packets to send, so only P12-P15 remain outstanding.

Phase 4 - timeout of P12: resend P12, P13, P14 and P15. All 4 arrive successfully, completing delivery of the 15 packets.

Original P1-P8	8 sends
Repeat P4-P8	5 sends
Original P9-P15	7 sends
Repeat P12-P15	4 sends

GBN originals = 8 + 7 = 15

GBN retransmissions = 5 + 4 = 9

Total GBN data transmissions = 15 + 9 = 24

The extra transmissions consist of the three actually lost originals plus six packets that were received out of order and discarded: P5, P6, P7, P13, P14 and P15. These six must be sent again because the GBN receiver did not save them.

Selective Repeat

The SR receiver buffers correctly received packets within its window and acknowledges them separately. Thus when P4 is missing, later valid packets such as P5, P6 and P7 need not be thrown away. Once P4 arrives, the receiver can release the consecutive buffered data. The same principle applies to later gaps.

Only the three missing packets require retransmission: P4, P8 and P12. Each is retried once under the successful-retransmission assumption. Buffered packets may wait for in-order delivery, but that waiting does not add a data transmission.

SR originals = 15

SR retransmissions = 3

Total SR data transmissions = 15 + 3 = 18

Comparison and interpretation

For this trace, SR saves 24 - 18 = 6 transmissions, or 6/24 = 25% relative to GBN. Useful-packet fractions are 15/24 = 62.5% for GBN and 15/18 = 83.3% for SR. These fractions compare transmission counts; they are not complete elapsed-time throughput calculations.

The problem does not specify ACK timing, timeout values or a recovery schedule, so a GBN total should not be quoted without assumptions. A different valid schedule can change how many later packets are outstanding when a loss is recovered. Interpreting every fourth attempt, including retransmissions, as lost would be a different loss model.

The general conclusion is independent of this example: GBN can repeat correctly received later packets, whereas SR saves and acknowledges them. SR's lower retransmission cost requires extra receiver buffering and more detailed acknowledgment and timer state.

Tutorial sheet 2

Q9. Divide 192.168.10.0/24 into four equal subnets

The original /24 network contains 32 - 24 = 8 host bits, giving 2^8 = 256 addresses. To obtain four equal subnets, borrow b bits so that 2^b = 4. Therefore b = 2 and the new prefix is /24 + 2 = /26.

New subnet mask = 255.255.255.192

Binary mask = 11111111.11111111.11111111.11000000

There are 6 host bits left in each subnet. Each block contains 2^6 = 64 addresses, conventionally giving 64 - 2 = 62 usable host addresses after excluding its network and broadcast addresses. The block increment in the last octet is 256 - 192 = 64.

Network, broadcast and usable range of every subnet

Subnet	Network	Usable host range	Broadcast
1 (/26)	.0	.1 - .62	.63
2 (/26)	.64	.65 - .126	.127
3 (/26)	.128	.129 - .190	.191
4 (/26)	.192	.193 - .254	.255

All addresses begin with 192.168.10

The first three octets in every table entry are 192.168.10. The four network addresses are therefore 192.168.10.0/26, 192.168.10.64/26, 192.168.10.128/26 and 192.168.10.192/26. The table abbreviates only the common prefix to keep the working compact.

Binary derivation of the last octet

Write the last octet as ss hhhhhh, where ss is the new subnet field and hhhhhh is the 6-bit host field. Set all host bits to zero for the network address and to one for the broadcast address.

ss = 00: 00 000000 = 0; 00 111111 = 63

ss = 01: 01 000000 = 64; 01 111111 = 127

ss = 10: 10 000000 = 128; 10 111111 = 191

ss = 11: 11 000000 = 192; 11 111111 = 255

For each subnet, first usable = network + 1, last usable = broadcast - 1. Example: the third subnet begins at .128, so .129 is its first host and .190 is its last host; .191 is the broadcast address. A host such as 192.168.10.150 belongs to this third /26 block.

Check: four blocks × 64 addresses = 256 addresses, with no overlap or gap. The usable count across the four subnets is 4 × 62 = 248. All four subnet blocks are usable in normal modern CIDR subnetting; do not discard the first or last subnet block using obsolete restrictions.

Q10. Smallest subnet for at least 50 usable hosts

Let h be the number of host bits. For an ordinary IPV4 LAN subnet, reserve one network address and one broadcast address, so the requirement is:

$2^h - 2 >= 50$

Try h = 5: 2^5 - 2 = 32 - 2 = 30 usable addresses. This is a /27 subnet and is too small. Try h = 6: 2^6 - 2 = 64 - 2 = 62 usable addresses, which meets the requirement.

Therefore the smallest address block that satisfies the requirement uses 6 host bits and prefix length 32 - 6 = /26. Its subnet mask is 255.255.255.192. Here "smallest subnet" means the fewest addresses in a block, which corresponds to the longest prefix that still provides enough usable hosts.

Example allocation: 192.168.10.0/26. Network address = 192.168.10.0; broadcast = 192.168.10.63; usable range = 192.168.10.1 through 192.168.10.62. Any of the /26 blocks found in Q9 can serve the department if its addresses are available.

Why a smaller block fails

A /27 contains only 32 total addresses, so even before considering other operational reservations it cannot provide 50 usable hosts. A /28 has just 14 usable addresses, and longer prefixes provide still fewer. A /25 gives 126 usable addresses but allocates more space than the stated minimum needs.

With /26, the mathematical spare usable capacity after 50 addresses is 62 - 50 = 12. If a gateway or other infrastructure address is additional to the 50 end devices, it also comes from the usable range; one extra gateway still leaves 61 addresses for endpoints. The network and broadcast addresses cannot be assigned to ordinary hosts.

The /31 point-to-point and /32 single-address special cases do not change this calculation for a 50-host LAN. Under the normal subnet rules used in the question, /26 is the required smallest suitable block.